RepOps — Reputation Operations Platform

Final Project Report

Karolis Živulskas

1. Introduction

RepOps (Reputation Operations) is a social media monitoring platform designed to detect hate speech and harmful content in Lithuanian political discourse on Facebook. The system automatically collects posts and comments from monitored Facebook pages and profiles, analyses them against configurable keyword sets, and surfaces flagged content for operator review through a web-based admin interface and real-time Grafana dashboards.

The motivation behind the project is the growing problem of coordinated political harassment and disinformation in Lithuanian online spaces, particularly in comment sections of news outlets and politician profiles. Manual moderation of these spaces is not scalable. RepOps automates the collection and triage pipeline, allowing a small team of operators to focus review effort on content most likely to be harmful.

2. System Architecture

The platform follows an event-driven microservice architecture built around a Celery task queue. The system is divided into three functional layers:

Collection layer — scrapes Facebook content via the Apify cloud scraping platform. Two Apify actors are used: facebook-posts-scraper to collect recent posts from a target page, and facebook-comments-scraper to collect comments on those posts. This approach avoids the need for browser automation or Facebook API credentials.

Analysis layer — processes each collected post through a keyword matching pipeline. Plain keywords are matched using an Aho-Corasick automaton, which scans the full post text in O(n) time regardless of the number of patterns. Regex patterns are matched separately for cases requiring word-boundary precision. Each match carries a severity level (1=low, 2=medium, 3=high), and the highest severity determines the post's overall score and label.

Presentation layer — a FastAPI web application serving both a browser-based admin UI (Jinja2 templates) and a REST API. Grafana dashboards connected to Prometheus (metrics) and Loki (logs) provide real-time operational visibility.

Data flow

```
Beat scheduler (every 15 min)
        │
        ▼
collect_all_active_targets
        │ fans out one task per target
```

```
        ▼
collect_target ──► Apify actors ──► posts + comments
        │
        │ enqueues one task per new post
        ▼
analyze_post ──► keyword matching ──► AnalysisResult saved
        │          (Aho-Corasick)       score + label
        │
        ▼
recalculate_profile ──► per-author risk score updated
        │
        ▼
Admin UI / Grafana ──► operator review
```

Post status lifecycle

Every collected post moves through a defined set of states:

Status	Meaning
`RAW`	Just collected, queued for analysis
`ANALYZED`	Analysis complete, score below threshold
`FLAGGED`	Score at or above threshold, awaiting review
`CLEARED`	Reviewed by operator, determined to be a false positive

---

3. Technology Stack

Layer	Technology	Purpose
Language	Python 3.12	Entire application REST API and admin UI Async task execution and scheduling Persistent storage Database access and migrations Facebook content collection O(n) multi-keyword matching Real-time dashboards Structured log aggregation Queue and worker visibility Service orchestration Cloud deployment
API framework	FastAPI + Uvicorn	
Task queue	Celery + Redis	
Database	PostgreSQL 16	
ORM	SQLAlchemy 2 + Alembic	
Scraping	Apify (httpx)	
NLP	pyahocorasick	
Metrics	Prometheus + Grafana	
Logging	structlog + Loki + Promtail	
Task monitoring	Celery Flower	
Containerisation	Docker + Docker Compose	
Infrastructure	Terraform (AWS EC2)	

4. Key Implementation Details

Keyword matching — Aho-Corasick automaton

Plain keyword patterns are compiled into an Aho-Corasick automaton (pyahocorasick) at analysis time. This algorithm builds a finite-state machine from all patterns and processes the input text in a single linear pass, making it significantly faster than running each pattern as a separate substring search. For a keyword set of size k and a post of length n, time complexity is O(n + k + m) where m is the number of matches, compared to O(n·k) for naive matching.

Regex patterns (marked is_regex=True in the database) use a separate word-boundary regex path (\bterm\b) for patterns where exact-word matching is required.

Profile risk scoring

Each Facebook user who has authored at least one collected post receives a risk score between 0 and 1, computed from four weighted signals:

Signal	Weight	Description
Flag rate	40%	Fraction of the author's posts that were flagged
Average score	35%	Mean analysis score across flagged posts
Severity signal	15%	Highest keyword severity seen (top / 3.0)
Volume bonus	10%	log(flagged + 1) / log(101) — rewards repeat offenders

A confidence factor (total_posts / (total_posts + 5)) dampens the flag rate for profiles with very few posts, preventing a single bad comment from producing a misleadingly high score. Scores are recalculated hourly and immediately after each post is analysed.

Observability

The API and worker each expose a Prometheus metrics endpoint. Because Celery forks multiple worker processes, the worker uses Prometheus's multiprocess mode (PROMETHEUS_MULTIPROC_DIR) so that counters incremented in child processes are aggregated before being scraped. All application logs are written as structured JSON using structlog, shipped to Loki via Promtail, and queryable in Grafana.

5. Deployment

The platform is deployed on AWS EC2 (eu-west-1, t3.medium) provisioned with Terraform. The Terraform configuration creates a VPC, public subnet, security group, EC2 instance running Ubuntu 22.04, and an Elastic IP for a stable public address.

The application stack runs entirely in Docker Compose with ten services. All service data (PostgreSQL, Redis, Prometheus, Loki, Grafana) is persisted in named Docker volumes.

Live deployment: http://54.74.132.233:8000/admin

6. Results

The live deployment has been running continuously and has collected the following data:

Monitored targets

Target	Type	Posts
LRT.lt	Facebook Page (Lithuanian national broadcaster)	**collected** 756
Remigijus Žemaitaitis	Public Profile (politician)	544 525
Šarūnas Černiauskas	Public Profile (journalist/politician)	

Collection summary

Metric	Count
Total items collected	1,825
— Posts	274
— Comments	1,551
Unique author profiles	1,053
Posts flagged for review	46 7
Posts cleared by operator	

Keyword configuration

Two Lithuanian keyword sets are configured with 20 patterns across three severity levels:

Set	Language	Low	Medium	High	Total
dešrinių troliai	Lithuanian	6	6	4	16
test-lt	Lithuanian	2	2	0	4

Top risk profiles

The profile scorer identified the following high-risk authors based on their posting history:

Author	Risk Score	Posts Analysed	Posts Flagged
Ram Une	0.606	3	2
Vincas Brazdžiūnas	0.564	1	1
Rasa Paulauskiene	0.564	1	1
Audrius Mejeris	0.564	1	1
Tautvydas Barstys	0.451	3	2

7. Conclusion

RepOps demonstrates an end-to-end pipeline for automated social media content monitoring, from scraping through analysis to operator review. The system successfully collected 1,825 items from three Lithuanian Facebook targets, flagged 46 posts for review, and maintains per-author risk profiles for 1,053 unique commenters.

The platform is production-deployed on AWS and fully observable through Grafana dashboards covering throughput, queue depth, worker health, and structured application logs. The keyword-based analysis approach is intentionally lightweight and transparent — operators can see exactly which patterns triggered a flag, making the system auditable and tunable without machine learning complexity.
Future extensions could include language detection to skip non-Lithuanian content, and expanding the monitored target list to cover additional pages in the Lithuanian political space.